

C++ Problem Solving

Algorithms/bellman_ford.cpp

```
#include <iostream>
#include <limits>
#include <stack>
#include <vector>

struct Edge { int from, to, weight; };

int main() {
    constexpr int N = 5;
    constexpr int INF = std::numeric_limits<int>::
        max() / 4;
    const std::vector<Edge> edges = {
        {0,1,6}, {0,2,7}, {1,2,8}, {1,3,-4}, {1,
4,5},
        {2,3,9}, {2,4,-3}, {3,0,2}, {3,4,7}, {4,
0,2}, {4,1,-2}
    };

    int source = 0;
    std::vector<int> dist(N, INF), previous(N,
-1);
    dist[source] = 0;

    for (int i = 0; i < N - 1; ++i) {
        bool changed = false;
        for (const auto& e : edges) {
            if (dist[e.from] != INF &&
dist[e.from] + e.weight < dist[e.to]) {
                dist[e.to] = dist[e.from] +
e.weight;
                previous[e.to] = e.from;
                changed = true;
            }
        }
        if (!changed) break;
    }
}
```

```

    for (const auto& e : edges)
        if (dist[e.from] != INF && dist[e.from]
+ e.weight < dist[e.to]) {
            std::cout << "Negative-weight cycle
detected.\n";
            return 0;
        }

    int target = 4;
    std::cout << "Shortest distance from " <<
source << " to " << target
        << ": " << dist[target] << '\n';

    std::stack<int> path;
    for (int v = target; v != -1; v = previous[v]
) path.push(v);
    std::cout << "Path: ";
    while (!path.empty()) { std::cout <<
path.top() << ' '; path.pop(); }
    std::cout << '\n';
}

```

Algorithms/edit_distance_suggestions.cpp

```
#include <algorithm>
#include <iostream>
#include <string>
#include <vector>

int editDistance(const std::string& a, const
    std::string& b) {
    const int n = static_cast<int>(a.size());
    const int m = static_cast<int>(b.size());
    std::vector<std::vector<int>> dp(n + 1,
        std::vector<int>(m + 1));

    for (int i = 0; i <= n; ++i) dp[i][0] = i;
    for (int j = 0; j <= m; ++j) dp[0][j] = j;

    for (int i = 1; i <= n; ++i) {
        for (int j = 1; j <= m; ++j) {
            if (a[i - 1] == b[j - 1])
                dp[i][j] = dp[i - 1][j - 1];
            else
                dp[i][j] = 1 + std::min({dp[i - 1][j],
                    dp[i][j - 1], dp[i - 1][j - 1]});
        }
    }
    return dp[n][m];
}

void closeSuggestions(const std::string& word,
    const std::vector<std::string>& dict) {
    bool found = false;
    for (const auto& candidate : dict) {
        if (editDistance(word, candidate) <= 2) {
            if (found) std::cout << ", ";
            std::cout << candidate;
            found = true;
        }
    }
    if (!found) std::cout << "No suggestions";
    std::cout << '\n';
}
```

```
}
```

```
int main() {  
    const std::vector<std::string> dict{  
        "apple", "application", "apply",  
        "banana", "band", "bandana", "cat",  
        "caterpillar"  
    };  
    std::string input;  
    std::cin >> input;  
    closeSuggestions(input, dict);  
}
```

Arrays/dynamic_matrix.cpp

```
#include <iostream>

int main() {
    int rows, cols;
    std::cout << "Enter the number of rows and
    columns: ";
    std::cin >> rows >> cols;
    if (rows <= 0 || cols <= 0) return 0;

    int** matrix = new int*[rows];
    for (int i = 0; i < rows; ++i) matrix[i] =
    new int[cols];

    std::cout << "Enter " << rows * cols << "
    integers for the matrix:\n";
    for (int i = 0; i < rows; ++i)
        for (int j = 0; j < cols; ++j)
            std::cin >> matrix[i][j];

    std::cout << "Matrix values:\n";
    for (int i = 0; i < rows; ++i) {
        for (int j = 0; j < cols; ++j) std::cout
        << matrix[i][j] << ' ';
        std::cout << '\n';
    }

    for (int i = 0; i < rows; ++i) delete[]
    matrix[i];
    delete[] matrix;
}
```

Data-Structures/task_device_assignment.cpp

```
#include <iostream>
#include <vector>

void tasksDevice(const std::vector<int>& tasks,
                 const std::vector<int>& device,
                 std::vector<int>& assigned) {
    std::vector<double> load(device.size(), 0.0);

    for (int task : tasks) {
        int chosen = 0;
        double minLoad = load[0] + static_cast<double>(task) / device[0];

        for (size_t j = 1; j < device.size(); ++j) {
            double newLoad = load[j] +
                static_cast<double>(task) / device[j];
            if (newLoad < minLoad) {
                minLoad = newLoad;
                chosen = static_cast<int>(j);
            }
        }

        load[chosen] += static_cast<double>(task) / device[chosen];
        ++assigned[chosen];
    }
}

int main() {
    int n, m;
    std::cin >> n;
    if (!std::cin || n < 0) return 1;
    std::vector<int> tasks(n);
    for (int& task : tasks) std::cin >> task;

    std::cin >> m;
    if (!std::cin || m <= 0) return 1;
    std::vector<int> device(m), assigned(m, 0);
```

```
    for (int& d : device) {
        std::cin >> d;
        if (!std::cin || d <= 0) return 1;
    }

    tasksDevice(tasks, device, assigned);
    for (int count : assigned) std::cout <<
count << ' ';
    std::cout << '\n';
}
```

Linked-List/linked_list_length.cpp

```
#include <iostream>

struct Node {
    int data;
    Node* next;
};

int main() {
    Node* head = new Node{2, nullptr};
    head->next = new Node{3, nullptr};
    head->next->next = new Node{4, nullptr};

    int length = 0;
    for (Node* current = head; current !=
        nullptr; current = current->next)
        ++length;

    std::cout << "Length of the linked list: "
    << length << '\n';

    while (head) {
        Node* next = head->next;
        delete head;
        head = next;
    }
}
```

OOP/site_classes.cpp

```
#include <iostream>
#include <string>
#include <utility>

class Web {
protected:
    std::string name;
    std::string url;

public:
    Web(std::string n, std::string u) :
        name(std::move(n)), url(std::move(u)) {}
    virtual void print() const {
        std::cout << name << '\n' << url << '\n';
    }
    virtual ~Web() = default;
};

class Site : public Web {
private:
    std::string ip;

public:
    Site(std::string n, std::string u,
        std::string address)
        : Web(std::move(n), std::move(u)),
        ip(std::move(address)) {}

    void print() const override {
        std::cout << name << '\n' << url << '\n'
        << ip << '\n';
    }
};

int main() {
    Web social("Facebook", "www.facebook.com");
    social.print();

    Site search("Site", "www.google.com",
```

```
    "35.44.67.101");  
    search.print();  
}
```

STL/sorted_pairs.cpp

```
#include <iostream>
#include <set>
#include <string>

int main() {
    std::set<std::pair<std::string, std::string>>
        fruits;

    for (int i = 0; i < 4; ++i) {
        std::pair<std::string, std::string>
            fruit;
        std::cout << "Enter fruit name: ";
        std::cin >> fruit.first;
        std::cout << "Enter color: ";
        std::cin >> fruit.second;
        fruits.insert(fruit);
    }

    std::cout << "Sorted pairs:\n";
    for (const auto& [fruit, color] : fruits)
        std::cout << fruit << " color: " <<
            color << '\n';
}
```

Strings/string_formatter.cpp

```
#include <algorithm>
#include <cctype>
#include <iostream>
#include <string>

class StringFormatter {
private:
    std::string str;

public:
    explicit StringFormatter(const std::string&
        initialStr) : str(initialStr) {}

    void reverse() { std::reverse(str.begin(),
        str.end()); }

    void toUpper() {
        std::transform(str.begin(), str.end(),
            str.begin(),
                [](unsigned char c) {
                    return static_cast<char>(std::toupper(c)); })
        ;
    }

    void toLower() {
        std::transform(str.begin(), str.end(),
            str.begin(),
                [](unsigned char c) {
                    return static_cast<char>(std::tolower(c)); })
        ;
    }

    void concatenate(const std::string& add) {
        str += add; }
    const std::string& getResult() const {
        return str; }
};

int main() {
```

```
std::string initial;  
std::cin >> initial;  
StringFormatter formatter(initial);  
formatter.toUpper();  
std::cout << formatter.getResult() << '\n';  
}
```